

**PART A: Peer Code Audit Checklist Table**

| Audit Category    | Specific Issue Located in Legacy Code                                                                              | Line Number(s) | Severity (Low / Med / High / Critical) | Impact / Risk                                                              |
|-------------------|--------------------------------------------------------------------------------------------------------------------|----------------|----------------------------------------|----------------------------------------------------------------------------|
| Global State      | Use of mutable global variables ( <code>dataArray</code> , <code>currentCount</code> , <code>maxCapacity</code> ). | Lines 8–10     | High                                   | Destroys modularity; prevents multiple array instances; thread-unsafe.     |
| Memory Leak       | Dynamic array resizing allocates new memory to <code>temp</code> without calling <code>delete[] dataArray</code> . | Lines 18–22    | Critical                               | Heap memory leak occurs on every capacity double; leads to RAM exhaustion. |
| Memory Leak       | Program terminates without executing <code>delete[] dataArray</code> .                                             | Line 100       | High                                   | Heap memory remains unreleased upon program execution completion.          |
| Out-of Bounds     | Traversal loop condition <code>i &lt;= currentCount</code> reads uninitialized/garbage heap memory.                | Line 58        | High                                   | Off-by-one buffer over-read; yields corrupted output or crash.             |
| Out-of Bounds     | <code>remove_item_at()</code> lacks upper bound validation checking ( <code>idx &gt;= currentCount</code> ).       | Line 31        | Critical                               | Attempting deletion at invalid indices causes array corruption or crash.   |
| Inefficient Shift | Redundant inner loop inside deletion shifting routine.                                                             | Lines 37–41    | Medium                                 | Artificially inflates element shift time complexity needlessly.            |

|                         |                                                                                                           |             |        |                                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------------------|-------------|--------|-------------------------------------------------------------------|
| <b>Naming Standards</b> | Inconsistent function naming ( <code>add_item</code> vs <code>findItem</code> vs <code>printAll</code> ). | Multiple    | Low    | Violates unified C++ naming conventions (camelCase / snake_case). |
| <b>Hardcoded Sizing</b> | Fixed matrix sizes (3x3) hardcoded in <code>processMatrix()</code> .                                      | Lines 66–68 | Medium | Prevents dynamic processing of multi dimensional array grids.     |
| Poor Modularity         | Functions rely on global variables instead of parameters.                                                 | Multiple    | High   | Makes functions difficult to reuse, test, and maintain.           |
| Missing Validation      | <code>add_item()</code> does not validate input values.                                                   | Lines 11–26 | Medium | Unexpected or invalid values can be inserted without checking.    |
| Missing Empty Check     | <code>remove_item_at()</code> does not check whether the array is empty.                                  | Lines 30–44 | Low    | Adds unnecessary code without improving                           |
| Incorrect Removal       | <code>currentCount</code> is decreased even when an invalid index is supplied.                            | Line 43     | High   | Can make the record count inconsistent with the actual data.      |
| Unnecessary Loop        | <code>findItem()</code> contains a dummy inner loop that executes only once.                              | Lines 49–51 | Low    | Adds unnecessary code without improving search performance        |

|                      |                                                                                               |         |     |                                                             |
|----------------------|-----------------------------------------------------------------------------------------------|---------|-----|-------------------------------------------------------------|
| Unused Variables     | Variables <code>r</code> and <code>c</code> are declared but never used.                      | Line 65 | Low | Creates unnecessary code and possible compiler warnings     |
| Poor Function Naming | <code>processMatrix()</code> does not clearly describe that the function transposes a matrix. | Line 65 | Low | Makes the function's purpose less clear to other developers |